

Installation Guide — Dragon Monster Game

Panduan lengkap untuk menginstal, mengkonfigurasi, dan menjalankan Dragon Monster Game Project di mesin lokal Anda.

Daftar Isi

1. Prasyarat Sistem
2. Instalasi Langkah demi Langkah
3. Konfigurasi Database
4. Menjalankan Aplikasi
5. Pengujian Endpoint API
6. Troubleshooting
7. Deployment ke Production

Prasyarat Sistem

Sebelum memulai, pastikan sistem Anda memenuhi persyaratan berikut:

Hardware Minimum

- RAM: 2 GB (4 GB disarankan)
- Disk Space: 500 MB untuk instalasi + dependensi Maven
- Processor: Multi-core 1.5 GHz atau lebih tinggi

Software yang Diperlukan

Komponen	Versi	Download
JDK (Java Development Kit)	25	oracle.com/java (https://www.oracle.com/java/technologies/downloads/)
Git	Latest	[git-scm.com] (https://git-scm.com/downloads)
PostgreSQL (opsional)	12+	[postgresql.org] (https://www.postgresql.org/download/)
Maven (opsional)	3.8+	Maven Wrapper sudah disediakan

Sistem Operasi yang Didukung

- macOS (10.14+)
- Linux (Ubuntu 18.04+, Debian 10+, CentOS 7+)
- Windows 10/11 (dengan PowerShell atau CMD)

Instalasi Langkah demi Langkah

Step 1: Verifikasi Instalasi JDK

Buka terminal/command prompt dan periksa versi Java:

```
```bash
java -version
javac -version
```
```

Output yang diharapkan:

```
```
```

```
openjdk version "25" 2024-XX-XX
OpenJDK Runtime Environment (build 25+XX)
OpenJDK 64-Bit Server VM (build 25+XX, mixed mode)
```
```

Jika JDK belum terinstal:

Windows

1. Download JDK 25 dari
[oracle.com] (<https://www.oracle.com/java/technologies/downloads/>)
2. Jalankan installer (`.exe`)
3. Pilih jalur instalasi (default: `C:\Program Files\Java\jdk-25`)
4. Klik "Next" hingga selesai
5. Set environment variable `JAVA\_HOME`:
 - Klik `Win + X` → "System"
 - "Advanced system settings" → "Environment Variables"
 - Klik "New" (System variables)
 - Variable name: `JAVA\_HOME`
 - Variable value: `C:\Program Files\Java\jdk-25`
 - Klik OK

macOS

```
```bash
brew tap homebrew/cask-versions
brew install java25
```
```

Atau download dari oracle.com dan jalankan installer.

Linux (Ubuntu/Debian)

```
```bash
sudo apt update
sudo apt install openjdk-25-jdk
```
```

Verifikasi setelah instalasi:

```
```bash
java -version
```
```

Step 2: Verifikasi Instalasi Git

```
```bash
git --version
```
```

Output yang diharapkan:

```
```
git version 2.40.0 (atau lebih tinggi)
```
```

Jika Git belum terinstal:

Windows

Download dan jalankan installer dari [git-scm.com] (<https://git-scm.com/downloads>)

macOS

```
```bash
brew install git
```
```

```
Linux (Ubuntu/Debian)
```bash
sudo apt install git
```
```

Step 3: Clone Repository

Pilih direktori untuk proyek, lalu clone repository:

```
```bash
Navigasi ke direktori yang diinginkan
cd ~/Projects
(atau di Windows: cd C:\Users\YourUsername\Projects)

Clone repository
git clone https://github.com/MangBujang/Java-Game-Project.git

Masuk ke direktori proyek
cd Java-Game-Project
```
```

Verifikasi struktur direktori:

```
```bash
ls -la
Anda akan melihat: pom.xml, mvnw, mvnw.cmd, src/, README.md, dll.
```
```

Step 4: Verifikasi Maven Wrapper

Repository sudah dilengkapi dengan Maven Wrapper, sehingga Anda tidak perlu menginstal Maven secara terpisah.

```
```bash
Di Windows
mvnw.cmd --version
```

```
Di macOS/Linux
./mvnw --version
```
```

Output yang diharapkan:

```
```
Apache Maven 3.8.1 (05ca950...)
Maven home: ~/.m2/wrapper/dists/apache-maven-3.8.1-bin/...
Java version: 25, vendor: Oracle Corporation
```
```

Konfigurasi Database

Aplikasi mendukung PostgreSQL untuk production dan H2 untuk development. Opsi 1:

Menggunakan PostgreSQL (Production)

Step 1: Instal PostgreSQL

Windows:

1. Download installer dari [postgresql.org] (<https://www.postgresql.org/download/windows/>)
2. Jalankan installer dan ikuti wizard
3. Ingat password untuk user `postgres`
4. Default port: `5432`

macOS:

```
```bash
brew install postgresql
brew services start postgresql
```
```

Linux (Ubuntu/Debian):

```
```bash
sudo apt install postgresql postgresql-contrib
sudo service postgresql start
```
```

Step 2: Buat Database dan User

```
```bash
Login ke PostgreSQL
psql -U postgres
```

```
Jalankan perintah berikut di psql:
CREATE DATABASE dragon_monster;
CREATE USER game_user WITH PASSWORD 'secure_password_123';
GRANT ALL PRIVILEGES ON DATABASE dragon_monster TO game_user;
\q
```
```

Step 3: Update `application.properties`

Edit file `src/main/resources/application.properties`:

```
```properties
Database Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/dragon_monster
spring.datasource.username=game_user
spring.datasource.password=secure_password_123
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
JPA/Hibernate
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

```
HikariCP Connection Pool
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.idle-timeout=30000
spring.datasource.hikari.connection-timeout=30000
```

```
Server Configuration server.port=8081
spring.application.name=Dragon Monster
```
```

Catatan Keamanan: Jangan commit kredensial ke repository publik. Gunakan environment variables (lihat bagian berikutnya).

Opsi 2: Menggunakan H2 (Development/Testing)

H2 adalah database in-memory yang cocok untuk development lokal.

Step 1: Uncomment H2 Dependency di `pom.xml`

Buka `pom.xml` dan cari baris berikut (sekitar line 53-57):

```
```xml
<!-- <dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
 <scope>runtime</scope>
</dependency> -->
```
```

Ubah menjadi:

```
```xml
<dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
 <scope>runtime</scope>
</dependency>
```
```

Step 2: Update `application.properties` untuk H2

```
```properties
H2 Database Configuration (In-Memory)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

JPA/Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

H2 Console (opsional, untuk debugging)
spring.h2.console.enabled=true

Server Configuration server.port=8081
spring.application.name=Dragon Monster
```
```

Step 3: Akses H2 Console (opsional)

Setelah aplikasi berjalan, akses H2 console di:

```
http://localhost:8081/h2-console
```

Konfigurasi Environment Variables (Recommended)

Untuk keamanan, gunakan environment variables instead of hardcoding credentials di file.

Step 1: Set Environment Variables

macOS/Linux

Edit `~/.bash\_profile` atau `~/.zshrc`:

```
``bash
export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/dragon_monster
export SPRING_DATASOURCE_USERNAME=game_user
export SPRING_DATASOURCE_PASSWORD=secure_password_123
export SERVER_PORT=8081
```
```

Kemudian reload:

```
``bash
source ~/.bash_profile
atau
source ~/.zshrc
```
```

Windows (PowerShell)

```
``powershell
$env:SPRING_DATASOURCE_URL = "jdbc:postgresql://localhost:5432/dragon_monster"
$env:SPRING_DATASOURCE_USERNAME = "game_user"
$env:SPRING_DATASOURCE_PASSWORD = "secure_password_123"
$env:SERVER_PORT = "8081"
```
```

Windows (Command Prompt)

```
``cmd
set SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/dragon_monster
set SPRING_DATASOURCE_USERNAME=game_user
set SPRING_DATASOURCE_PASSWORD=secure_password_123
set SERVER_PORT=8081
```
```

Step 2: Update `application.properties` Update

agar Spring membaca environment variables:

```
``properties
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/
dragon_monster}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:postgres}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:}
server.port=${SERVER_PORT:8081}
```

```
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
}
```

Menjalankan Aplikasi

Opsi 1: Menjalankan dengan Maven Wrapper (Development)

```
macOS/Linux
```bash
./mvnw spring-boot:run
```
```

```
Windows (PowerShell)
>`powershell
>`.\mvnw.cmd spring-boot:run
```

```
Windows (Command Prompt)
>`>`cmd
mvnw.cmd spring-boot:run
```

Output yang diharapkan:

[illegible]

```
2024-07-23 10:30:45.123 INFO 12345 [main] c.d.g.DragonMonsterApplication :
Starting DragonMonsterApplication
```

```
...
2024-07-23 10:30:50.456 INFO 12345 [main] c.d.g.DragonMonsterApplication : Started
DragonMonsterApplication in 5.123 seconds (JVM running for 5.456)
...
```

Server berjalan di `http://localhost:8081`

Opsi 2: Build dan Jalankan JAR

```
Build Project
```bash
 macOS/Linux
 ./mvnw clean package -DskipTests
```

```
Windows
mvnw.cmd clean package -DskipTests
```

Output: `target/dragon-monster-0.0.1-SNAPSHOT.jar`

```
Jalankan JAR
```bash
java -jar target/dragon-monster-0.0.1-SNAPSHOT.jar
```
```

Atau dengan environment variables:

```
```bash
export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/dragon_monster
export SPRING_DATASOURCE_USERNAME=game_user
export SPRING_DATASOURCE_PASSWORD=secure_password_123

java -jar target/dragon-monster-0.0.1-SNAPSHOT.jar
```
```

Opsi 3: Menjalankan dengan IDE

Menggunakan IntelliJ IDEA

1. Buka IntelliJ IDEA
2. File → Open → Pilih folder `Java-Game-Project`
3. Tunggu indexing selesai
4. Klik kanan pada `DragonMonsterApplication.java`
5. Pilih "Run 'DragonMonsterApplication'"
6. Atau tekan `Ctrl+Shift+F10` (Windows/Linux) atau `Cmd+Shift+R` (macOS)

Menggunakan VS Code + Extension Pack for Java

1. Install extensions:
  - Extension Pack for Java
  - Maven for Java
  - Spring Boot Extension Pack
2. Buka folder proyek
3. Klik pada `DragonMonsterApplication.java`
4. Klik "Run" di atas method `main()`



Pengujian Endpoint API

Setelah aplikasi berjalan, test endpoint menggunakan curl, Postman, atau Insomnia.

1. Test GET /api/heroes

```
```bash
curl -X GET http://localhost:8081/api/heroes \
-H "Content-Type: application/json"
```
```

Response yang diharapkan:

```
```json
[
  {
    "id": 1,
    "name": "Wanderer Magician",
    "level": 1,
    "health": 100,
  }
]
```

```
    "mana": 50,  
    "attack": 15,  
    "defense": 10  
  }  
]  
...
```

2. Test GET /api/dragons

```
```bash  
curl -X GET http://localhost:8081/api/dragons \
 -H "Content-Type: application/json" \
...`
```

## 3. Test POST /api/combat/hero-attack

```
```bash  
curl -X POST http://localhost:8081/api/combat/hero-attack \  
  -H "Content-Type: application/json" \  
  -d '{  
    "heroId": 1,  
    "enemyId": 1,  
    "attackType": "normal"  
  }'  
...`
```

4. Test GET /api/slots (Save Slots)

```
```bash  
curl -X GET http://localhost:8081/api/slots \
 -H "Content-Type: application/json" \
...`
```

## 5. Test POST /api/slots (Create Save Slot)

```
```bash  
curl -X POST http://localhost:8081/api/slots \  
  -H "Content-Type: application/json" \  
  -d '{  
    "playerName": "John Doe",  
    "level": 5,  
    "experience": 1000,  
    "heroId": 1  
  }'  
...`
```

Menggunakan Postman

1. Download [Postman] (<https://www.postman.com/downloads/>)

2. Buat Collection baru: "Dragon Monster Game"
3. Tambahkan requests:
 - GET `http://localhost:8081/api/heroes`
 - GET `http://localhost:8081/api/dragons`
 - GET `http://localhost:8081/api/slots`
 - POST `http://localhost:8081/api/slots`
 - POST `http://localhost:8081/api/combat/hero-attack`
4. Click "Send"

Akses Frontend

Buka browser dan akses halaman game:

Halaman	URL
Game Utama	`http://localhost:8081/game.html`
Map 1	`http://localhost:8081/map.html`
Map 2	`http://localhost:8081/map2.html`
Map 3	`http://localhost:8081/map3.html`
Setup	`http://localhost:8081/setup.html`

Troubleshooting

Masalah 1: "JDK 25 tidak ditemukan"

Solusi:

```
```bash
```

```
java -version
```

Jika tidak menunjukkan versi 25, install JDK 25 terlebih dahulu

Atau set JAVA\_HOME ke path JDK 25

```
```
```

Windows:

```
```cmd
```

```
set JAVA_HOME=C:\Program Files\Java\jdk-25
```

```
echo %JAVA_HOME%
```

```
```
```

macOS/Linux:

```
```bash
```

```
export JAVA_HOME=/path/to/jdk-25
```

```
echo $JAVA_HOME
```

```
```
```

Masalah 2: "Port 8081 sudah digunakan"

Solusi:

Ubah port di `application.properties`:

```
```properties
```

```
server.port=8082
```

```
```
```

Atau jalankan dengan environment variable:

```
```bash
export SERVER_PORT=8082
./mvnw spring-boot:run
```
```

Masalah 3: "Database connection refused"

Solusi:

1. Verifikasi PostgreSQL berjalan:

```
```bash
```

macOS

```
brew services list
```

Linux

```
sudo service postgresql status
```

Windows

Check Services → PostgreSQL

```
```
```

2. Verifikasi credentials di `application.properties`

3. Pastikan database sudah dibuat:

```
```bash
```

```
psql -U postgres -c "SELECT datname FROM pg_database WHERE
datname='dragon_monster';"
```

```
```
```

Masalah 4: "Maven Wrapper tidak executable"

Solusi (macOS/Linux):

```
```bash
```

```
chmod +x mvnw
```

```
./mvnw --version
```

```
```
```

Masalah 5: "Cannot resolve symbol 'SpringApplication'"

Solusi:

```
```bash
```

Clear cache Maven

```
./mvnw clean
```

Download dependencies

```
./mvnw dependency:resolve
```

Rebuild

```
./mvnw compile
```

```
```
```

Deployment ke Production

Deploy ke Heroku

Step 1: Buat Akun Heroku

Daftar di [heroku.com] (<https://www.heroku.com/>)

Step 2: Instal Heroku CLI

```
```bash
```

macOS

```
brew tap heroku/brew && brew install heroku
```

Windows

Download installer dari <https://devcenter.heroku.com/articles/heroku-cli>

Linux

```
curl https://cli-assets.heroku.com/install-ubuntu.sh | sh
```

```
```
```

Step 3: Login ke Heroku

```
```bash
```

```
heroku login
```

```
```
```

Step 4: Create Heroku App

```
```bash
```

```
heroku create your-app-name
```

```
```
```

Step 5: Add PostgreSQL Add-on

```
```bash
```

```
heroku addons:create heroku-postgresql:hobby-dev
```

```
```
```

Step 6: Deploy

```
```bash
```

```
git push heroku main
```

```
```
```

Step 7: View Logs

```
```bash
```

```
heroku logs --tail
```

```
```
```

Deploy ke Railway.app

Step 1: Buat Akun di Railway

Daftar di [railway.app] (<https://railway.app/>)

Step 2: Hubungkan Repository GitHub

1. Login ke Railway
2. Klik "New Project"
3. Pilih "Deploy from GitHub repo"
4. Authorize GitHub dan pilih repository

Step 3: Add PostgreSQL Database

1. Klik "Add Service"
2. Pilih "PostgreSQL"
3. Railway akan auto-configure environment variables

Step 4: Deploy
Railway akan otomatis deploy dari main branch

Deploy ke AWS EC2

Step 1: Launch EC2 Instance

1. Buka AWS Console
2. EC2 → Launch Instance
3. Pilih Ubuntu 22.04 LTS
4. Instance type: t3.micro (free tier)
5. Security Group: Allow port 8081, 5432

Step 2: SSH ke Instance

```
```bash
ssh -i your-key.pem ubuntu@your-instance-ip
```
```

Step 3: Install Java dan PostgreSQL

```
```bash
sudo apt update
sudo apt install openjdk-25-jdk postgresql postgresql-contrib
```

Start PostgreSQL

```
sudo service postgresql start
```

Setup database (same as local)

```
sudo -u postgres psql
CREATE DATABASE dragon_monster;
CREATE USER game_user WITH PASSWORD 'password';
GRANT ALL PRIVILEGES ON DATABASE dragon_monster TO game_user;
\q
```
```

Step 4: Clone Repository dan Deploy

```
```bash
git clone https://github.com/MangBujang/Java-Game-Project.git
cd Java-Game-Project
```

```
./mvnw clean package -DskipTests
```

Run dengan environment variables

```
export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/dragon_monster
export SPRING_DATASOURCE_USERNAME=game_user
export SPRING_DATASOURCE_PASSWORD=password
```

```
nohup java -jar target/dragon-monster-0.0.1-SNAPSHOT.jar > app.log 2>&1 &
```
```

Referensi Tambahan

- [Spring Boot Official Documentation] (<https://spring.io/projects/spring-boot>)
- [PostgreSQL Documentation] (<https://www.postgresql.org/docs/>)
- [Maven Documentation] (<https://maven.apache.org/>)
- [H2 Database Documentation] (<https://www.h2database.com/>)

Checklist Instalasi

- ☐ JDK 25 sudah terinstal dan verified
- ☐ Git sudah terinstal
- ☐ Repository sudah di-clone
- ☐ PostgreSQL sudah terinstal (atau H2 dependency uncommented)
- ☐ Database sudah dibuat
- ☐ `application.properties` sudah dikonfigurasi
- ☐ Environment variables sudah di-set (opsional tapi recommended)
- ☐ Aplikasi bisa dijalankan tanpa error
- ☐ API endpoints bisa dites dengan curl/Postman
- ☐ Frontend halaman bisa diakses di browser

Jika ada pertanyaan atau masalah, silakan buat issue di GitHub repository atau hubungi developer.

Selamat menggunakan Dragon Monster Game!